

Control de Posición de Motor DC

Curtale Lautaro Adrián, Torreblanca Agustín

UTN FRA - Técnicas Digitales III

Avellaneda, Argentina

lacurtale@gmail.com

agusalejotorre00@gmail.com

Abstract—El presente proyecto consiste en el control, por medio de PID, de un motor de DC. Mediante el PID se regula el PWM que alimenta a dicho motor, logrando controlar su movimiento en relación con su posición actual medida. La medición es realizada mediante un encoder magnético acoplado a uno de los ejes del motor. Por último, se visualizan dichas mediciones por medio de una interfaz gráfica web desarrollada en un sistema basado en Linux integrado en una Raspberry Pi 1, que cuenta con el sistema operativo Raspberry Pi Os Lite, la cual intercambia información mediante el protocolo UART con una Raspberry Pi Pico 2W, que recopila los datos del encoder. Se programó en lenguaje C tanto el módulo de kernel v6.12.47 al que se accedió con SSH como el microcontrolador con el entorno de desarrollo Visual Studio Code v1.102.3 con la extensión Raspberry Pi Pico v0.18.0. El desarrollo de la aplicación web se utilizó Python con la biblioteca Flask.

Palabras clave: Motor CC, UART, PWM, Encoder, SSH, freeRTOS, Python, Raspberry Pi.

I. ÍNDICE

I. ÍNDICE	1
II. FUNDAMENTACIÓN	1
A. Problemática	1
B. Fundamentación Teórica de Hardware	1
C. Fundamentación Teórica de Software	1
III. OBJETIVOS	1
IV. ALCANCE	1
V. DIAGRAMA EN BLOQUES	2
VI. DIAGRAMA DE SOFTWARE	2
VII. DIAGRAMA DE TAREAS	2
VII. COMANDOS IMPLEMENTADOS	2
A. Comandos GET	2
B. Comandos SET	2
IX. CONCLUSIONES	3
X. REFERENCIAS	3
Anexo 1 - Diagrama en bloques	4
Anexo 2 - Diagrama de Software	5
Anexo 3 - Diagrama de Tareas	6
Anexo 4 - Aplicación Consola	7

II. FUNDAMENTACIÓN

A. Problemática

En varias aplicaciones industriales o de automatización, tales como brazos robóticos, impresoras 3D o máquinas CNC, es necesario el control de la posición de un motor, esto con el fin de poder mantener una posición constante frente a perturbaciones del sistema y/o variaciones en la carga, lo cual es clave para aplicaciones donde la posición del motor es crítica.

Además, es necesario hoy en día poder manejar estos aspectos de forma remota, pudiendo interactuar con los mismos y configurarlos a distancia, ahorrando costos y tiempo. Por eso, se propone usar un sistema embebido de Linux en una Raspberry Pi 1 para lograr la modificación del proyecto trabajado en la EGA.

B. Fundamentación Teórica de Hardware

No hubo modificaciones del hardware presentado en la EGA, se agrega una Raspberry Pi 1 [1], la cuál está conectada por cable Ethernet a una Notebook, y también estará conectada mediante cables a los pines que de destinaron para la UART en la EGA con la Raspberry Pi Pico 2W.

C. Fundamentación Teórica de Software

La ampliación de la EGB (el desarrollo del módulo del kernel) se desarrolló sobre el sistema operativo Raspberry Pi OS Lite [2], el cual cuenta con un kernel de Linux en la versión 6.12.47 [3], esto implica desarrollar un driver para el puerto UART y su posterior programación para lograr comunicar el kernel space con el user space. Este kernel space implementa un device tree junto un char device para manejar el puerto, mientras que en el user space se escribe y se lee sobre el char device para así poder interactuar por UART. La gestión remota del sistema operativo y la ejecución de la CLI se realizan a través del protocolo SSH sobre Ethernet, asegurando un canal de comunicación seguro y estándar para el envío de comandos.

III. OBJETIVOS

Los objetivos para la EGB fueron:

- Desarrollar un driver para conectar la UART
- Desarrollar una aplicación gráfica y una aplicación por consola para interactuar con la EGA
- Uso de device tree, coordinar programas con kernel space y user space

UTN FRA – TORREBLANCA – CURTALE – Control de Posición de Motor de CC - Informe Final de EGB

IV. ALCANCE

- Implementar una aplicación de consola.
- Implementar una aplicación con interfaz gráfica que pueda graficar el error y el ángulo.
- Modificar los valores de Set Point, Tipo de Entrada, Pendiente si es necesario y Banda de Error.
- Leer los valores de PWM, Error, Ángulo, Set Point, Tipo de Entrada, Pendiente y Banda de Error.
- Habilitar o deshabilitar el control PID de forma remota.

V. DIAGRAMA EN BLOQUES

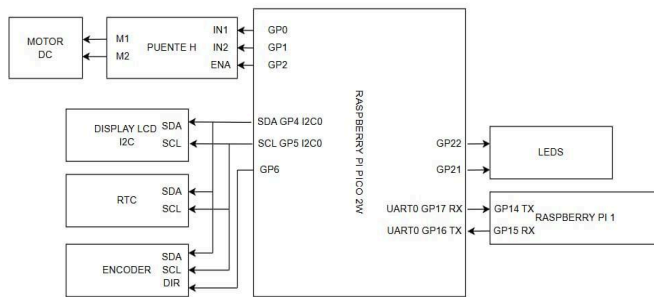


Fig.1 Diagrama en bloques

VI. DIAGRAMA DE SOFTWARE

En la figura 2 se muestra el diagrama del software.

Se relaciona el RTOS (Para nuestro proyector FreeRTOS [4]) que corre en la Raspberry Pi Pico 2W [5] a través del protocolo de comunicación UART con los pines dedicados para dicho uso en la planificación de la EGA con el GPOS, accederemos mediante un módulo del Kernel al kernel space el cual genera un char device para la comunicación con el user space y un driver para manejar la comunicación mediante UART con la Pi Pico 2W.

Se implementó una app de consola la cual le permite al usuario:

- Seleccionar la acción que quiere realizar (configurar, leer o salir)
- Seleccionar la variable a modificar/consultar entre las mencionadas en los alcances del proyecto.
- Leer o modificar el archivo
- Muestra el comando que se envía mediante UART a la Raspberry Pi Pico 2W.

La aplicación web se implementó mediante la biblioteca Flask para reducir el uso de RAM y recursos, en la misma:

- Se genera una página web y un servidor.
- En la parte superior se escriben los valores que se van censando cada 200 ms.
- En la parte inferior se presentan unas opciones para modificar valores de Set Point, Tipo de Entrada, Pendiente y Banda de Error, un botón para modificar todo a la vez, y un botón para habilitar y deshabilitar el control PID..
- Abajo de todo se presenta una gráfica a tiempo real del error y del ángulo actual, estas se van renovando con nuevos valores cada 200 ms.

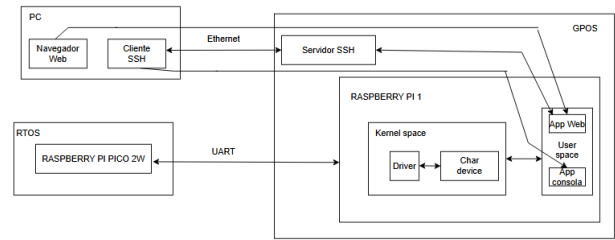


Fig.2 Diagrama de Software

VII. DIAGRAMA DE TAREAS

En el Anexo 3 se muestra el diagrama de tareas de la EGB.

Se implementó en RTOS un software para la EGB que cuenta con 9 tareas, a comparación de las 8 tareas que existían en la EGA.

La diferencia radica en la creación de una tarea `task_uart`, la cual es la encargada de controlar el puerto de la UART en la Raspberry Pi Pico 2W, esta se encarga de recibir los comandos, cargar las nuevas configuraciones en las colas correspondientes, y enviar los datos a leer por las aplicaciones creadas en la Raspberry Pi 1.

El resto de las tareas mantienen sus funcionamientos sin modificaciones, exceptuando la tarea de `task_flags`, en esta tarea, la cual se encarga del control de los leds y la banda de error, se añadió una variable a la estructura de configuración llamada Banda de Error, para poder hacer este campo editable. Por esto, se tuvo que modificar levemente su funcionamiento para compararlo directamente a la variable en lugar de que sea a un número fijo.

En las tareas de `pid_control` y teclado, se agregaron llamadas a las nuevas colas creadas para `task_uart`, para leer los datos actuales en caso de que se hayan modificado.

VII. COMANDOS IMPLEMENTADOS

A. Comandos GET

Se plantearon los siguientes comandos para leer variables:

- `get spo`: obtiene el valor del set point en grados
- `get tip`: obtiene el tipo de entrada.
- `get pen`: obtiene el valor de la pendiente en porcentaje.
- `get bde`: obtiene el valor de la banda de error en grados.
- `get ang`: obtiene el valor del ángulo actual en grados.
- `get pwm`: obtiene el valor del duty cycle del pwm (en porcentaje)
- `get err`: obtiene el valor del error en grados.

B. Comandos SET

Se plantearon los siguientes comandos para modificar variables:

- `set spo xxx`: se ingresa un valor set point de 0° a 360°
- `set tip x`: define el valor del tipo de entrada, 0 para escalón y 1 para rampa.

UTN FRA – TORREBLANCA – CURTALE – Control de Posición de Motor de CC - Informe Final de EGB

- set pen xxx: se ingresa un valor de pendiente del 1% al 100%.
- set bde xx: se ingresa un valor de 1° al 20° para la banda de error
- set ena x: define si el PID se encuentra activo, 0 para desactivado y 1 para activar

IX. CONCLUSIONES

Al haber trabajado con una Raspberry Pi más antigua que la que propuso la cátedra (la Raspberry Pi 4), surgieron diversos desafíos y conflictos a la hora de la resolución de la EGB.

Durante la integración del sistema, se identificó una inestabilidad crítica en la comunicación remota vía Ethernet, caracterizada por interrupciones sistemáticas en la sesión SSH. Para resolver este problema, se tuvo que hacer una reconfiguración se realizó una reconfiguración del servicio de red (dhcpd.conf). Se implementó una IP estática, eliminando la dependencia de routers y garantizando un canal de comando robusto y continuo para el User Space.

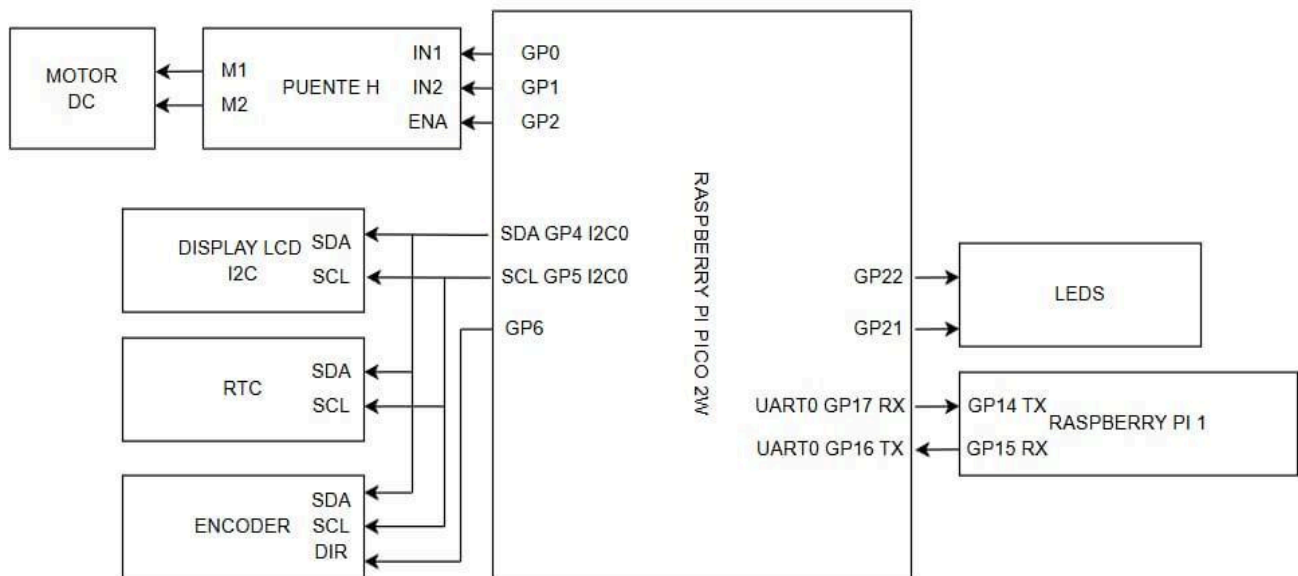
La arquitectura del procesador BCM2835 de la Raspberry Pi 1 y la limitada memoria RAM disponible presentaron un desafío para la implementación de la interfaz gráfica. Para resolver esto sin comprometer el rendimiento del Kernel Space, se optó por una arquitectura de software Cliente-Servidor optimizada utilizando la librería Flask. Esta decisión de diseño permitió minimizar el uso de recursos de CPU y RAM en la Raspberry Pi 1, delegando la carga computacional del renderizado al cliente web y asegurando que los recursos del CPU estuvieran disponibles para la ejecución prioritaria del driver UART.

Referido a la utilización del sistema operativo de uso general, el mismo efectúa de forma exitosa el alcance propuesto en consenso con la cátedra, permitiendo el monitoreo y modificación de variables de forma remota, e implementando gráficos a tiempo real de variables del proyecto. Además, el gráfico de error nos permite observar el funcionamiento del control PID para la corrección del PWM de forma más precisa, lo cual no se podía hacer previamente de un modo no empírico.

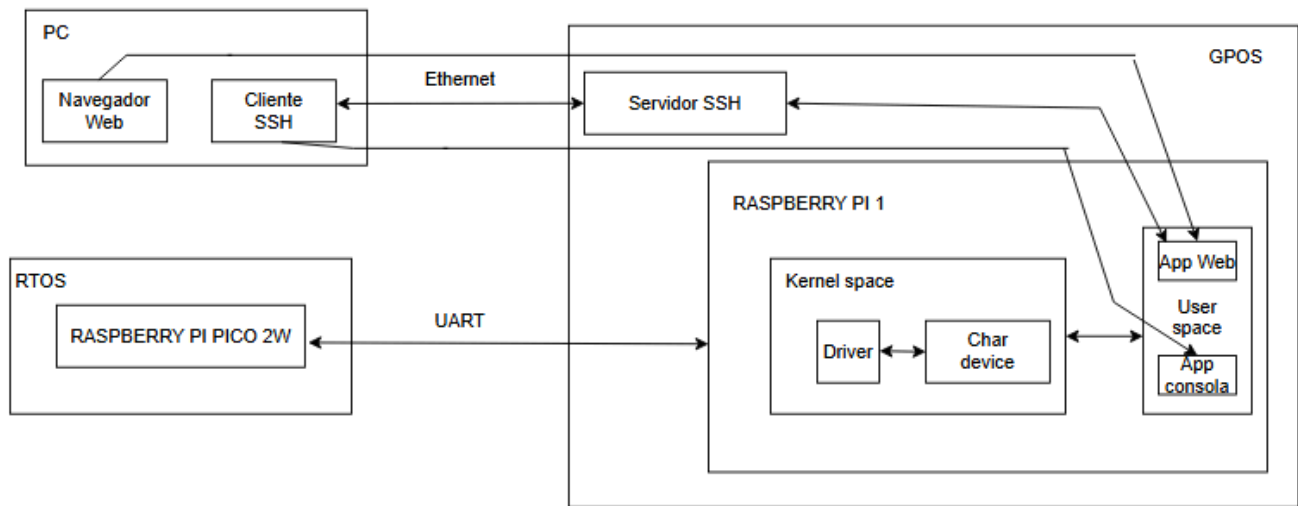
X. REFERENCIAS

- [1] Raspberry Pi 1:
<https://pip-assets.raspberrypi.com/categories/892-raspberrypi-5/documents/RP-008370-DS-1-rp1-peripherals.pdf?disposition=inline>
- [2] Raspberry Pi OS:
<https://www.raspberrypi.com/software/operating-systems/>
- [3] Kernel Linux v6.12.47:
<https://elixir.bootlin.com/linux/v6.12.47/source>
- [4] FreeRTOS:
<https://www.freertos.org/>
- [5] Raspberry Pi Pico 2W:
<https://pip-assets.raspberrypi.com/categories/1088-raspberrypi-pico-2-w/documents/RP-008304-DS-1-pico-2-w-datasheet.pdf?disposition=inline>

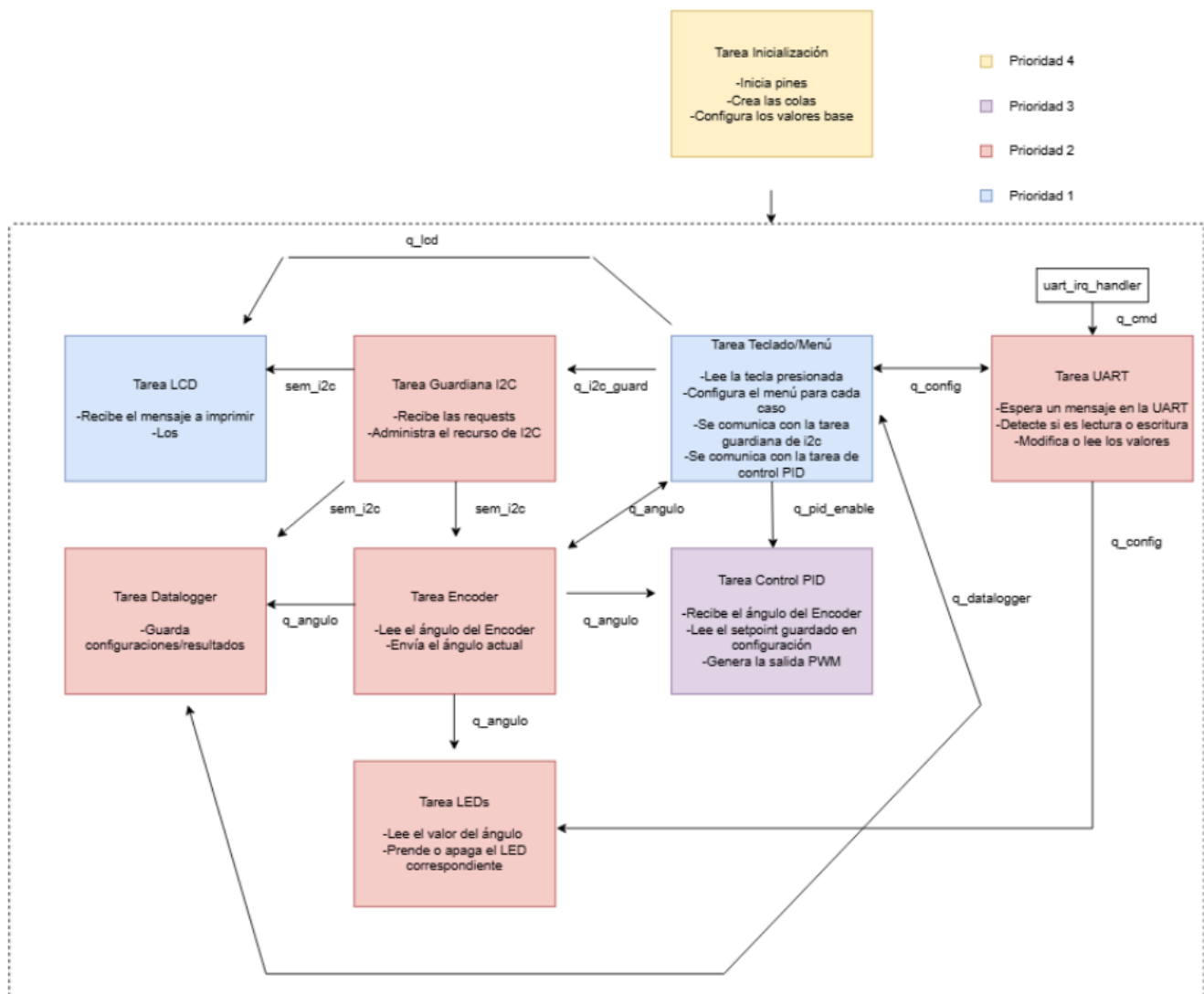
Anexo 1 - Diagrama en bloques



Anexo 2 - Diagrama de Software



Anexo 3 - Diagrama de Tareas



Anexo 4 - Aplicación Consola

 lautaro@lautaro: ~/4_apps/cli

```
-----{ Aplicación UART POS_CTRL }-----  
-----  
Ingrese accion a realizar:  
-----  
1> Enviar configuración (SET)  
2> Obtener información (GET)  
3> Salir  
-----  
Opción [1-3]: █
```

Anexo 5 - Aplicación Web

Control de Posición de Motor DC

Tipo de Señal Rampa	Pendiente [%] 4	PWM [%] 0
Set Point [°] 210	Ángulo Actual [°] 209	Error [°] 1

Setpoint

Enviar Set Point

Tipo de Entrada

Rampa

Enviar Tipo de Entrada

Pendiente

Enviar Pendiente

Banda de Error

Enviar Banda

Enviar Configuración Completa

ON/OFF

Respuesta: Set ok

Error de Posición

